

Cấu trúc dữ liệu và giải thuật

Bài 1: Ôn tập cấu trúc và con trỏ

GV: Nguyễn Hữu Phát

Nội dung

1. Nhập/ xuất dữ liệu
2. Khái niệm và định nghĩa kiểu cấu trúc
3. Khai báo và các thao tác truy xuất các thành phần biến cấu trúc.
4. Mảng cấu trúc.

Xuất dữ liệu

- Xuất dữ liệu ra màn hình với hàm printf
 - Cú pháp: printf("chuỗi định dạng", bt1, bt2...);
 - Ví dụ:

```
printf("Gia tri cua x la : %6.2f",x);
```
- Xuất dữ liệu ra tập tin với hàm fprintf
 - Cú pháp: fprintf(tên con trỏ FILE, "chuỗi định dạng", bt1, bt2...);
 - Ví dụ:

```
FILE* f;  
f = fopen("data.txt", "w");  
fprintf(f, "Gia tri cua x la : %.2f",x);
```

Nhập dữ liệu

- Nhập dữ liệu từ bàn phím với hàm scanf:
 - Cú pháp: `scanf("chuỗi định dạng",&biến1,&biến2...);`
 - Ví dụ:
`scanf("%d%d",&x,&y);`
- Nhập dữ liệu từ tập tin với hàm fscanf:
 - Cú pháp: `fscanf(tên con trỏ FILE, "chuỗi định dạng",&biến1,&biến2...);`
 - Ví dụ:
`FILE* f;`
`f = fopen("data.txt", "r");`
`fscanf(f, "%d", &x);`

Nhập xuất dữ liệu

- Chuỗi định dạng qui định
 - Phải nằm trong dấu nháy kép “ ”
 - Có bao nhiêu biến phải có bấy nhiêu định dạng
 - Thứ tự định dạng phải phù hợp với danh sách biến
 - Mã định dạng phải phù hợp với kiểu dữ liệu của biến
 - Mỗi mã định dạng bắt đầu bằng dấu %
 - Danh sách biến qui định:
 - Các biến phải phân cách bằng dấu phẩy
 - Giá trị của biến phải phù hợp với mã định dạng
 - Riêng với lệnh scanf thì trước các biến phải có ký hiệu &

Nhập xuất dữ liệu

Mã định dạng	Ý nghĩa
%3d	In số nguyên (int) có độ dài 3 ký tự
%4ld	In số nguyên (long) có độ dài 4 ký tự
%.2f	In số thực (float) có 2 số lẻ (phần nguyên không qui định)
%5.3lf	In số thực (double) có 3 số lẻ, phần nguyên có độ dài là 5.
%o	In số nguyên hệ 8
%x	In số nguyên hệ 16
%c	In ký tự
%s	In chuỗi ký tự
%e hoặc %E	In số thực dạng mũ, VD : 1.134e+01

Nhập xuất dữ liệu

CHUỖI ESCAPE

<u>Tổ hợp</u>	<u>Tên gọi</u>	<u>Diễn giải</u>
\a	Alert	Phát âm thanh ra loa
\b	Backspace	Lùi con nháy 1 vị trí
\f	Form feed	Sang trang kế tiếp
\n	New line	Sang dòng mới
\r	Carriage return	Đưa con trỏ về đầu dòng
\t	Horizontal tab	Di chuyển con trỏ tới vị trí tab kế tiếp
\\	Backslash	Ký tự '\'
\'	Single quote	Ký tự dấu nháy đơn
\"	Double quote	Ký tự dấu nháy đôi
\?	Question mark	Ký tự dấu hỏi
\<octal digit>		Hằng hệ 8
\<hexa digit>		Hằng hệ 16

Khái niệm và định nghĩa kiểu cấu trúc

- Cấu trúc (Struct) là một cấu trúc dữ liệu, mỗi thể hiện bao gồm một số các phần tử dữ liệu cùng hoặc không cùng kiểu, được nhóm lại với nhau.
- Quan sát danh sách sinh viên sau

Họ và tên	Tuổi	Điểm TB
Nguyễn Văn A	18	7.3
Nguyễn Văn B	20	7.0
Nguyễn Văn C	21	9.2
Nguyễn Văn D	18	5.6
...		

Định nghĩa kiểu struct

```
typedef struct StructName  
{  
    Khai báo các biến thành phần;  
} StructAlias;
```

Ví dụ:

```
typedef struct SinhVien  
{  
    char hoten[30];  
    int  tuoi;  
    float diemtb;  
} sinhvien;
```

Khai báo và truy xuất các thành phần biến cấu trúc

- <Tên biến kiểu struct>.<Tên biến thành phần>
- Ví dụ:
 - sinhvien.hoten
 - sinhvien.diemtb

VD: Truy xuất các phần tử struct

```
#include <conio.h>
```

```
#include <stdio.h>
```

```
typedef struct SINHVIEN
```

```
{ char Hoten[15];
```

```
    int Tuoi;
```

```
} SINHVIEN;
```

```
int main()
```

```
{
```

```
    SINHVIEN sv;
```

```
    printf("Nhap ho ten:");
```

```
    gets(sv.Hoten);
```

```
    printf("Nhap tuoi: ");
```

```
    scanf("%d", &sv.Tuoi);
```

```
    printf("%20s %3d", sv.Hoten, sv.Tuoi);
```

```
}
```

Mảng cấu trúc

```
typedef struct SINHVIEN  
{  
    int tuoi;  
    char ten[20];  
} SINHVIEN;  
  
SINHVIEN ds[20];
```

Ví dụ mảng cấu trúc

- Chương trình nhập N quyển sách gồm các thông tin: Tên sách, số lượng,... Xuất danh mục sách ra màn hình.

Ví dụ mảng cấu trúc

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#define MAX 3
```

```
typedef struct Sach
```

```
{
```

```
    char tenSach[30];
```

```
    int soluong;
```

```
} Sach;
```

Ví dụ mảng cấu trúc

```
int main()
{
    // Khai báo mảng cấu trúc, gồm nhiều
    // phần tử mang kiểu Sách
    Sach arrSach[MAX];

    // Nhập thông tin các sách có trong
    // mảng
    for (int i=0; i<MAX; i++)
    {
        char ch;
        printf("Nhap ten sach:");
        gets(arrSach[i].tenSach);
        printf("Nhap so luong sach:");
        scanf("%d",&arrSach[i].soluong);
        while (ch=getchar() != '\n')
            printf("\n");
    }

    // Xuất thông tin sách có trong mảng
    for (int i=0; i<MAX; i++)
    {
        printf("\n Sach %s co %d
               quyen",arrSach[i].tenSach,arrSach[
               i].soluong);
    }
}
```

Bài tập

- Tạo một cấu trúc SINHVIEN gồm các thông tin sau: Maso, HoTen, diem1, diem2, diem3, diemTB.
- Viết chương trình
 - Nhập danh sách N sinh viên với số N nhập vào từ bàn phím
 - Tính điểm trung bình của từng sinh viên
 - Xuất danh sách sinh viên với điểm trung bình ra màn hình

Mã nguồn

```
#include <conio.h>
#include <stdio.h>

typedef struct SINHVIEN
{
    int maso;
    char hoten[20];
    float diem1, diem2, diem3,
    diemTB;
} SINHVIEN;

void nhapDanhSach(SINHVIEN sv[],
int n);
void tinhDiemTB(SINHVIEN sv[], int
n);
void xuatDanhSach(SINHVIEN sv[],
int n);
int main()
{
    int n=0;
    printf("\n Nhap so luong sinh
    vien: ");
    scanf("%d", &n);

    SINHVIEN sv[n];
    nhapDanhSach(sv, n);
    tinhDiemTB(sv, n);
    xuatDanhSach(sv, n);
}
```

Mã nguồn

```
void nhapDanhSach(SINHVIEN sv[],
int n)
{
    for (int i=0; i<n; i++)
    {
        char ch=' ';

        printf("\n Nhap ma so sinh
vien: ");
        scanf("%d", &sv[i].maso);
        while (ch=getchar() != '\n');
        printf("\n Nhap ho ten sinh
vien: ");
        gets(sv[i].hoten);
        printf("\n Nhap diem 1: ");
        scanf("%f", &sv[i].diem1);
        printf("\n Nhap diem 2: ");
        scanf("%f", &sv[i].diem2);
        printf("\n Nhap diem 3: ");
        scanf("%f", &sv[i].diem3);
    }
}
```

```
void tinhDiemTB(SINHVIEN sv[], int
n)
{
    for (int i=0; i<n; i++)
    {
        sv[i].diemTB = (sv[i].diem1 +
sv[i].diem2 + sv[i].diem3)/3;
    }
}

void xuatDanhSach(SINHVIEN sv[],
int n)
{
    for (int i=0; i<n; i++)
    {
        printf("\n Maso: %5d \t Ho
ten: %20s \t Diem 1: %.2f \t Diem 2:
%.2f \t Diem 3: %.2f",
            sv[i].maso, sv[i].hoten,
sv[i].diem1, sv[i].diem2, sv[i].diem3,
sv[i].diemTB);
    }
}
```

Kết quả thực thi chương trình

```
D:\OneDrive - Mahidol University\Desktop\Untitled1.exe

Nhap so luong sinh vien: 2
Nhap ma so sinh vien: 234
Nhap ho ten sinh vien: Huu Phat
Nhap diem 1: 4.5
Nhap diem 2: 6.7
Nhap diem 3: 8.3
Nhap ma so sinh vien: 789
Nhap ho ten sinh vien: Truong Hoang
Nhap diem 1: 3.2
Nhap diem 2: 7.5
Nhap diem 3: 9.2

Maso:   234   Ho ten:      Huu Phat   Diem 1: 4.50   Diem 2: 6.70   Diem 3: 8.30
Maso:   789   Ho ten:      Truong Hoang   Diem 1: 3.20   Diem 2: 7.50   Diem 3: 9.20
-----
Process exited after 19.26 seconds with return value 0
Press any key to continue . . .
```

Con trỏ

- Khái niệm, khai báo và truy xuất biến con trỏ.
- Con trỏ và kiểu cấu trúc.
- Con trỏ và mảng 1 chiều.

Khái niệm, khai báo và truy xuất biến con trỏ

- Con trỏ là một biến chứa địa chỉ vùng nhớ của một biến khác
- Con trỏ được sử dụng trong chương trình để truy cập bộ nhớ và vận dụng các bộ nhớ
- Chúng ta đã sử dụng các địa chỉ bộ nhớ như là tham số cho hàm `scanf()`
 - `scanf("%d", &value)`: lưu 1 giá trị (được nhập) vào 1 địa chỉ cụ thể trong bộ nhớ
 - `value` là biến và `&value` là địa chỉ

Khái niệm con trỏ

- Con trỏ là một biến mà giữ địa chỉ bộ nhớ của biến khác.
- Một biến con trỏ phải giữ địa chỉ của một biến cùng loại với biến con trỏ đó.

Khai báo biến con trỏ

- Khai báo: Cú pháp: <kiểu dữ liệu>* <tên biến con trỏ>
 - <kiểu dữ liệu>: một kiểu dữ liệu hợp lệ bất kỳ (int, long, char ...)
 - Ví dụ: int i, *p;
- Gán giá trị:
 - p = &i; /* tham chiếu đến địa chỉ i */
 - p = 0;
 - P = NULL; /* tương đương với p = 0; */

Truy xuất biến con trỏ

- Sử dụng biến con trỏ phải qua 3 bước:
 - Khai báo biến con trỏ
 - Khởi tạo giá trị cho biến con trỏ
 - Sử dụng biến con trỏ
- Toán tử “address-of operator” (&) dùng để **lấy địa chỉ** của một biến.
- Toán tử “dereferenced pointer” (*) dùng để **lấy nội dung** tại địa chỉ mà biến con trỏ đang trỏ vào.

Sử dụng con trỏ

- Để trả về nhiều hơn một giá trị từ một hàm
- Thuận tiện hơn trong việc truyền các mảng và chuỗi từ một hàm đến một hàm khác
- Sử dụng con trỏ để làm việc với các phần tử của mảng thay vì truy xuất trực tiếp vào các phần tử này
- Để cấp phát bộ nhớ động và truy xuất vào vùng nhớ được cấp phát này (dynamic memory allocation)

Con trỏ và kiểu cấu trúc

- Chúng ta có thể truy xuất struct bằng cách sử dụng con trỏ chỉ đến struct đó.
- Truy xuất đến phần tử của con trỏ struct bằng toán tử ->

Con trỏ và kiểu cấu trúc

```
#include <conio.h>
#include <stdio.h>

typedef struct
{
    char Hoten[15],
    int Tuoi;
} SINHVIEN;

int main()
{
    SINHVIEN* sv = new SINHVIEN;
    //Biến sv mang kiểu SINHVIEN, là biến con trỏ
    printf("Nhap ho ten:");
    gets(sv->Hoten);
    printf("Nhap tuoi: ");
    scanf("%d", &sv->Tuoi);

    printf("%20s %3d", sv->Hoten, sv->Tuoi);
}
```

Con trỏ và mảng 1 chiều

- Mảng bản chất là một địa chỉ hoặc giá trị con trỏ
- Con trỏ và mảng gần như giống hệt nhau trong việc truy xuất bộ nhớ
- Sự khác biệt
 - Con trỏ là một biến có giá trị là địa chỉ
 - Mảng là một địa chỉ cố định cụ thể có thể được coi như là một con trỏ hằng
 - Khi một mảng được khai báo, trình biên dịch phải phân bổ một địa chỉ cơ sở và một lượng vừa đủ dung lượng lưu trữ để chứa tất cả các yếu tố của mảng.
 - Địa chỉ cơ sở của mảng là vị trí ban đầu trong bộ nhớ nơi mảng được lưu trữ

Con trỏ và mảng 1 chiều (tt)

```
Int a[100], *p;
```

→ Giả sử biến a được cấp địa chỉ 300; lần lượt các ô nhớ ở những địa chỉ 304, 308, ..., 696 được dành để lưu các giá trị của a[0], a[1], a[2], ..., a[99]

```
p = a;
```

```
p = &a[0];
```

→ hai cách khai báo tương đương nhau và gán tham chiếu ô nhớ 300 cho biến con trỏ p

```
a = p;
```

```
++a;
```

```
a += 2;
```

→ Không cho phép

Con trỏ và mảng 1 chiều (tt)

- Truy cập địa chỉ các phần tử mảng:

`array[100];` // array là mảng 1 chiều có 100 phần tử

- Địa chỉ của phần tử đầu tiên trong mảng: `&array[0]` hay `array`
- Địa chỉ của phần tử thứ hai trong mảng: `&array[1]` hay `array + 1`
- Tổng quát, địa chỉ của phần tử thứ i trong mảng: `&array[i]` hay `array + i`
- `array[i]` và `*(array + i)` đều biểu diễn nội dung của địa chỉ đó, nghĩa là, giá trị của phần tử thứ i trong mảng array

Con trỏ và mảng 1 chiều (tt)

- Chương trình sau đây biểu diễn mối quan hệ giữa các phần tử mảng và địa chỉ của chúng

```
#include<stdio.h>
void main()
{
    static int array[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int i;
    for (i = 0; i < 10; i ++)
    {
        printf("i=%d , array[i]=%d , *(array+i)= %d ", i, array[i], *(array+i));
        printf("&array[i] = %X , array + i = %X", &array[i], array + i);
    }
}
```

Con trỏ và mảng 1 chiều (tt)

i=0	array[i]=1	*(array+i)=1	&array[i]=194	array+i = 194
i=1	array[i]=2	*(array+i)=2	&array[i]=196	array+i = 196
i=2	array[i]=3	*(array+i)=3	&array[i]=198	array+i = 198
i=3	array[i]=4	*(array+i)=4	&array[i]=19A	array+i = 19A
i=4	array[i]=5	*(array+i)=5	&array[i]=19C	array+i = 19C
i=5	array[i]=6	*(array+i)=6	&array[i]=19E	array+i = 19E
i=6	array[i]=7	*(array+i)=7	&array[i]=1A0	array+i = 1A0
i=7	array[i]=8	*(array+i)=8	&array[i]=1A2	array+i = 1A2
i=8	array[i]=9	*(array+i)=9	&array[i]=1A4	array+i = 1A4
i=9	array[i]=10	*(array+i)=10	&array[i]=1A6	array+i = 1A6

- Kết quả trình bày rõ ràng sự khác nhau giữa:
 - array[i] - biểu diễn giá trị của phần tử thứ i trong mảng
 - &array[i] - biểu diễn địa chỉ của nó.